

MODULE 06/09

React Native & Expo

Créer des apps mobiles avec JavaScript

■ Durée : 1 semaine

■ Projet : Social Seller

■ Niveau : Débutant →
Opérationnel

Objectifs de la semaine

- ✓ Comprendre le modèle de composants React et le rendu déclaratif
 - ✓ Utiliser les hooks essentiels : useState, useEffect, useCallback, useRef
 - ✓ Construire des listes performantes avec FlatList
 - ✓ Maîtriser StyleSheet pour créer des interfaces cohérentes
 - ✓ Créer une app Expo fonctionnelle avec navigation basique
-

Planning de la semaine

JOUR 1	Composants et JSX	2-3h
• React Native vs React web : View au lieu de div, Text au lieu de p • JSX : écrire du HTML dans du JavaScript • Props : passer des données d'un composant parent à enfant • Composants fonctionnels : la seule façon moderne		
JOUR 2	State et hooks	2-3h
• useState : gérer un état local (valeur + setter) • useEffect : déclencher un effet quand l'état change • useCallback : mémoriser une fonction pour éviter les re-renders • useRef : référencer un élément sans déclencher de re-render		
JOUR 3	Listes et interactions	2-3h
• FlatList : afficher de longues listes de manière performante • FlatList inverted : scroll automatique en bas (comme un chat) • Pressable, TouchableOpacity — gérer les taps • Pull-to-refresh : RefreshControl		
JOUR 4	StyleSheet et layouts	2-3h
• StyleSheet.create() : styles typés et optimisés • Flexbox en React Native : flex, flexDirection, justifyContent, alignItems • Dimensions, Platform.OS : adapter selon iOS/Android • KeyboardAvoidingView : éviter que le clavier cache les inputs		
JOUR 5	Mini-projet de la semaine	3-4h
• Créer une app 'Liste de courses' avec Expo • FlatList avec items, checkbox pour cocher un item		

- TextInput pour ajouter un nouvel item
- Animation Animated.Value sur les items cochés (opacité)
- Pull-to-refresh qui simule un rechargement depuis une API

Concepts clés détaillés

Composant de base

```
import { View, Text, Pressable, StyleSheet } from 'react-native';

// Props typées avec TypeScript
interface ConversationCardProps {
  customerName: string;
  lastMessage: string | null;
  unreadCount: number;
  onPress: () => void;
}

export function ConversationCard({
  customerName, lastMessage, unreadCount, onPress
}: ConversationCardProps) {
  return (
    <Pressable style={styles.card} onPress={onPress}>
      <Text style={styles.name}>{customerName}</Text>
      <Text style={styles.preview} numberOfLines={1}>
        {lastMessage ?? 'Aucun message'}
      </Text>
      {unreadCount > 0 && (
        <View style={styles.badge}>
          <Text style={styles.badgeText}>{unreadCount}</Text>
        </View>
      )}
    </Pressable>
  );
}
```

useState et useEffect

```
import { useState, useEffect, useCallback } from 'react';
```

```

export default function InboxScreen() {
  const [conversations, setConversations] = useState<Conversation[]>([]);
  const [isLoading, setIsLoading] = useState(true);

  // useCallback : mémoriser la fonction pour éviter
  // qu'elle soit recréée à chaque render
  const fetchConversations = useCallback(async () => {
    const { data } = await supabase.from('conversations').select('*');
    setConversations(data ?? []);
  }, []); // [] = dépendances : ne change jamais

  // useEffect : s'exécute après le premier rendu
  useEffect(() => {
    fetchConversations().finally(() => setIsLoading(false));
  }, [fetchConversations]);

  // Nettoyage : important pour les subscriptions/timers
  useEffect(() => {
    const timer = setInterval(() => { /* ... */ }, 1000);
    return () => clearInterval(timer); // Cleanup
  }, []);

  // ...
}

```

FlatList — la liste performante

```

<FlatList
  data={conversations}
  keyExtractor={(item) => item.id}
  renderItem={({ item }) => (
    <ConversationCard
      customerName={item.customer_name ?? 'Client'}
      lastMessage={item.lastMessage}
      unreadCount={item.unreadCount}
      onPress={() => router.push(`/conversation/${item.id}`)}
    />
  )}
  refreshControl={
    <RefreshControl refreshing={isLoading} onRefresh={handleRefresh} />
  }

```

```
}  
ListEmptyComponent={<EmptyState />}  
ListHeaderComponent={<Text style={styles.title}>Inbox</Text>}  
// inverted=true pour un chat (scroll en bas)  
// inverted  
/>
```

StyleSheet — le CSS de React Native

```
const styles = StyleSheet.create({  
  // Flexbox est la base de tous les layouts  
  container: {  
    flex: 1, // Prendre tout l'espace disponible  
    backgroundColor: '#F8FAFC',  
  },  
  row: {  
    flexDirection: 'row', // Horizontal  
    alignItems: 'center', // Centrer verticalement  
    justifyContent: 'space-between',  
    padding: 16,  
    backgroundColor: 'FFFFFF',  
    borderRadius: 14,  
    marginBottom: 10,  
  },  
  badge: {  
    minWidth: 20,  
    height: 20,  
    borderRadius: 10,  
    backgroundColor: '#6366F1', // Indigo Social Seller  
    alignItems: 'center',  
    justifyContent: 'center',  
    paddingHorizontal: 6,  
  },  
});
```

■ React Native n'utilise pas de CSS. `StyleSheet.create()` compile les styles en IDs optimisés côté natif. Flexbox est disponible partout — pas besoin de grilles, floats, ou positions CSS. La direction par défaut est column (vertical) contrairement au web (row).

Exercices pratiques

Exercice 1 — Premier composant (30 min)

- Créer un composant ProductCard avec : nom, prix, stock, badge 'Stock bas' si stock < 5
- Le rendre cliquable avec Pressable
- Passer les props depuis un écran parent

Exercice 2 — Liste dynamique (45 min)

- Créer un écran avec FlatList de produits (données statiques en dur)
- Ajouter un champ de recherche TextInput qui filtre la liste
- Ajouter un état vide élégant si aucun résultat

Exercice 3 — Animation (30 min)

- Créer un skeleton loader animé (Animated.Value avec pulse)
 - Afficher les skeletons pendant 2 secondes, puis remplacer par des vraies données
-

Ressources pour aller plus loin

- React Native docs : reactnative.dev/docs
- Expo docs : docs.expo.dev
- React hooks guide : react.dev/reference/react
- Expo SDK API : docs.expo.dev/versions/latest